Chapter 12 — Networked Programs (Sockets, HTTP, urllib, Beautiful Soup)

Practice Sheet — Worked Examples, Tasks, Challenges & Deliverable Project

---

Section A — Worked Examples (Solved, Study Karo Pehle)

Worked Example 1 — Raw Socket Se Text File Fetch Karna

Goal: Manual `socket` library use karke `romeo.txt` fetch karo aur sirf pehli 5 lines print karo.

```python
```

```python
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))

cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

full_data = b''
while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    full_data += data

mysock.close()

# Response ko headers aur body mein split karo (blank line separator)
header_part, _, body_part = full_data.partition(b'\r\n\r\n')

print("--- HEADERS ---")
print(header_part.decode())

print("--- FIRST 5 LINES OF BODY ---")
lines = body_part.decode().splitlines()
for line in lines[:5]:
    print(line)
```

Key Learning Points:

- `recv(512)` chunks mein data deta hai — isliye `full_data` mein accumulate karna parta hai agar poora response chahiye.

- Headers aur body ke darmiyan separator `\r\n\r\n` hota hai — `partition()` se easily split ho jata hai.
- `.decode()` sirf ek dafa, poore accumulated bytes pe karna zyada efficient hai (baar baar chunk-wise decode karne se multi-byte character break ho sakta hai).

Worked Example 2 — `urllib` Se Word Frequency Counter (Top 5 Words)

Goal: `urllib.request` se URL open karo, word counts nikaalo, aur top 5 most frequent words print karo (sorted).

```python
import urllib.request

url = 'http://data.pr4e.org/romeo.txt'
fhand = urllib.request.urlopen(url)

counts = dict()
for line in fhand:
    words = line.decode().split()
    for word in words:
        counts[word] = counts.get(word, 0) + 1

# Dictionary ko count ke hisaab se sort karo (descending)
sorted_counts = sorted(counts.items(), key=lambda pair: pair[1], reverse=True

print("Top 5 words:")
for word, count in sorted_counts[:5]:
    print(f"{word} -> {count}")
```

Key Learning Points:

- `dict.items()` se `(key, value)` tuples milte hain jinhe `sorted()` ke sath `key=lambda` se sort kiya ja sakta hai.
- `counts.get(word, 0) + 1` pattern — dictionary counting ka standard idiom hai, isay yaad rakho.
- Har `line` bytes hoti hai jab tak `.decode()` na ho.

Worked Example 3 — Beautiful Soup Se Links Extract Aur Filter Karna

Goal: Kisi page se saare anchor tags nikaalo, lekin sirf wo links print karo jo `http` se start hote hain (relative links ko skip karo).

```python
```

```
import ssl
from urllib.request import urlopen
from bs4 import BeautifulSoup

# SSL certificate verification bypass (HTTPS sites ke liye zaroori)
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

url = 'https://www.si.umich.edu/'
html = urlopen(url, context=ctx).read()
soup = BeautifulSoup(html, 'html.parser')

tags = soup('a')
valid_links = []

for tag in tags:
    href = tag.get('href', None)
    if href is not None and href.startswith('http'):
        valid_links.append(href)

print(f"Total anchor tags: {len(tags)}")
print(f"Valid (absolute) links: {len(valid_links)}")
for link in valid_links[:10]:
    print(link)
```

Key Learning Points:

- `tag.get('href', None)` safe hai — agar `href` attribute missing ho to crash nahi hoga, `None` milega.
- `href.startswith('http')` se relative links (`/about.html` jese) filter ho jate hain.
- SSL context sirf HTTPS sites ke liye chahiye — HTTP sites ke liye zaroorat nahi.

Section B — Practice Tasks (Tumhe Solve Karni Hain)

Practice Task 1 — Socket Se HTTP Headers Nikaalo

Raw `socket` library use karke `data.pr4e.org` port 80 se `romeo.txt` GET karo. Response ko headers aur body mein separate karo, aur sirf headers ka dictionary banao (e.g. `{'Content-Type': 'text/plain', 'Content-Length': '...'}`) — har header line ko `:` pe split karke.

Hint: header lines is format mein hoti hain: `Key: Value`

Practice Task 2 — urllib Se Line Count Aur Character Count

`urllib.request.urlopen()` se `http://data.pr4e.org/romeo.txt` open karo aur ye 3 cheezein calculate karo:

1. Total lines kitni hain
2. Total words kitne hain
3. Total characters kitne hain (newlines/whitespace excluding)

Output format: `Lines: X, Words: Y, Characters: Z`

Practice Task 3 — Beautiful Soup Se Sirf Image Tags Nikaalo

`https://www.si.umich.edu/` (ya koi bhi page) se Beautiful Soup use karke saare `<img>` tags nikaalo aur unke `src` attribute print karo. Agar kisi img tag ka `alt` attribute bhi ho to wo bhi sath mein print karo (format: `src -> alt_text`).

Hint: `soup('img')` *anchor tags ki tarah hi kaam karta hai, bas tag name different hai.*

Section C — Challenges (Zyada Mushkil, Multiple Concepts Combine)

Challenge 1 — Retry Logic Wala Socket Client

Raw socket code likho jo agar `connect()` fail ho jaye (host down ya port band ho), to usay silently crash hone ke bajaye:

- Error catch karo (`try/except` with `socket.error`)
- Ek friendly message print karo: `"Connection failed: <error details>"`
- 3 baar retry karo (2 second gap ke sath, `time.sleep()` use karke) before giving up completely

---

Challenge 2 — Mini Word Frequency Comparator (2 URLs)

Do alag URLs (jese do different text files) se word counts nikaalo aur compare karo:

- Wo words jo dono files mein common hain (kitni baar dono mein aaye, side by side)
- Wo words jo sirf pehli file mein hain
- Wo words jo sirf doosri file mein hain

Output ek clean formatted report ki tarah ho, e.g.:

```
Common words: 12
Only in File 1: 8
Only in File 2: 5
```

Hint: Python `set()` operations (`intersection`, `difference`) yahan bohat kaam aayenge.

Challenge 3 — Depth-1 Mini Web Crawler

Ek page se start karo, Beautiful Soup se uske saare `http`-based links nikaalo, aur phir un links mein se **sirf pehle 3 links** ko bhi visit karo (yani depth-1 crawl). Har visited page ka:

- URL
- Us page pe kitne anchor tags mile

...ek dictionary ya list mein collect karke print karo. Duplicate URLs dobara visit mat karo (ek `set()` mein visited URLs track karo).

Caution: Sirf apne diye gaye test URLs (jese dr-chuck.com pages) pe try karo — random websites ko heavily crawl mat karo (ethical scraping yaad rakho — Part 7 ki notes dekho).

Section D — Deliverable Project

Project: "Mini Link Health Checker"

Scenario: Tumhe ek chhota tool banana hai jo kisi bhi webpage ke saare links check kare ke wo "reachable" hain ya nahi — bilkul jese ek website admin apne broken links dhoondta hai.

Requirements:

1. User se ek URL input lo (`input()` se).
2. `urllib` + `BeautifulSoup` use karke us page ke saare anchor tags (`<a>`) nikaalo aur unke `href` values collect karo.
3. Sirf absolute links (jo `http` se start hote hain) ko filter karo — relative links skip karo.
4. Har link ke liye:
   - Us link ko `urlopen()` se open karne ki koshish karo (with SSL context aur ek short timeout, e.g. `urlopen(link, context=ctx, timeout=5)`)
   - Agar successfully open ho jaye → status `"OK"` mark karo
   - Agar koi error aaye (URLError, timeout, etc.) → status `"BROKEN"` mark karo, aur error ka short reason bhi save karo
5. Final summary report print karo is format mein:

```
Link Health Report for: <original URL>
Total links checked: N

[OK]     http://example.com/page1
[BROKEN] http://example.com/page2  -> reason: timed out
[OK]     http://example.com/page3
...

Summary: X working, Y broken
```

6. **Bonus (optional):** Report ko ek `.txt` file mein bhi save karo (`link_health_report.txt`), taake user isay baad mein dobara dekh sake.

Concepts Being Tested (Cumulative):

- `urllib.request.urlopen()` aur error handling (`try/except`)
- Beautiful Soup se HTML parsing aur tag/attribute extraction
- SSL context handling for HTTPS
- String filtering (`.startswith()`)
- Dictionary/list-based data collection aur formatted report generation
- File writing (bonus part)

Suggested Test URLs:

- `http://www.dr-chuck.com/page1.htm` (simple, 1 link)
- `http://www.dr-chuck.com/` (homepage, multiple links — kuch broken bhi mil sakte hain, jo tumhare tool ko test karne ka acha case hai)

Submission Checklist

- [ ] Worked Examples samajh liye (run karke dekho, tweak karo)
- [ ] Practice Task 1, 2, 3 complete
- [ ] Challenge 1, 2, 3 complete
- [ ] Deliverable Project complete aur test URLs pe run kiya
- [ ] (Optional) Bonus file-writing feature add ki

Good luck — code likho, error aayein to unhe read karo (Python errors kaafi descriptive hote hain), aur incrementally test karo (chhote steps mein print statements dalo)!